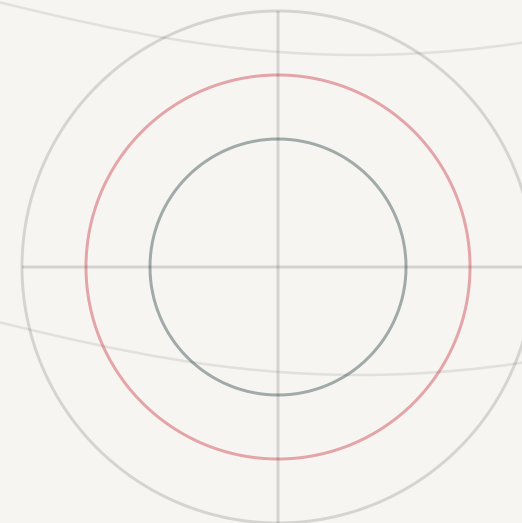


MODULO SVILUPPO WEB · PROGETTO FINALE

Il mio sito personale: le scelte dietro al progetto

Un sito in HTML, CSS e Sass costruito per raccontare le due anime del mio lavoro, comunicazione e sport. In queste pagine spiego perché ho fatto certe scelte, tecniche e di design, e non altre.



RENATO VALIGI

Digital & Sports Communication Strategist

Autore Renato Valigi

Progetto Sviluppo Web · Sito personale

01 Il brief e gli obiettivi

02 Struttura del sito

03 Identità visiva: palette e font

04 HTML e CSS, quasi senza JavaScript

05 Il menu mobile, solo CSS

06 Sass, non solo CSS

07 Bootstrap, ma solo l'essenziale

08 Mobile-first e responsive

09 Accessibilità e SEO

10 Il form contatti ed EmailJS

11 Pubblicazione su GitHub Pages

12 Dal brief al sito online, e i link

IL PUNTO DI PARTENZA

«Un sito personale che raccontasse chi sono, in HTML e CSS, quasi senza scorciatoie.»

COSA CHIEDEVA IL BRIEF

- Almeno cinque pagine collegate tra loro: home, progetti, CV, blog, contatti
- Il CV scritto direttamente in HTML, non caricato come PDF
- Contenuti reali, non segnaposto: progetti veri e articoli scritti da me
- Un sito responsive, testato su più dispositivi
- Solo HTML e CSS, con EmailJS come unica eccezione facoltativa per il form contatti
- Pubblicazione online, con un repository pubblico su GitHub

Perché questo approccio

Ho trattato il vincolo dell'HTML e CSS puri come un'occasione più che come un limite: dimostrare che un sito curato, accessibile e animato quanto basta si può costruire anche senza appoggiarsi a un framework JavaScript.

STRUTTURA

Cinque pagine, tre articoli, un solo filo conduttore

Ogni pagina ha un compito preciso: la home presenta, progetti e CV dimostrano, il blog racconta, i contatti chiudono il cerchio.

Nessuna pagina è lì solo per riempire il menu.

```
renato-valigi/  
├─ assets/  
│   ├─ scss/          sorgenti Sass  
│   ├─ css/style.css  CSS compilato  
│   ├─ img/           foto, favicon  
│   ├─ js/script.js   unico script  
│   └─ docs/          PDF progetti  
├─ blog/              tre articoli  
├─ index.html  
├─ progetti.html  
├─ cv.html  
├─ blog.html  
├─ contatti.html  
└─ .nojekyll
```

- **Home** — presentazione e doppia identità, comunicazione e sport
- **Progetti** — portfolio con PDF scaricabili, lavori veri del master
- **Chi sono / CV** — percorso professionale, scritto in HTML puro
- **Blog** — tre articoli scritti da me, uno già uscito su una rivista di coaching
- **Contatti** — form con validazione nativa ed EmailJS opzionale

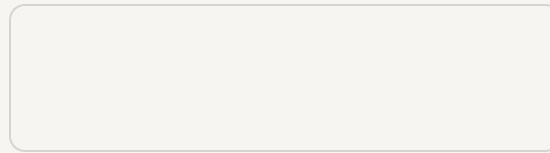
IDENTITÀ VISIVA

Una palette che racconta due anime

Il rosso è il colore Grypho, comunicazione e creatività. Il verde petrolio è il lato analitico, dati e strategia. Li ho tenuti entrambi, usati con parsimonia per non fare un arcobaleno.



Ink · #14110f
Testo, sfondi scuri



Paper · #f7f5f2
Sfondo principale



Rosso Grypho · #c1121f
Comunicazione, creatività



Verde petrolio · #1f3a3d
Dati, strategia, coaching

Fraunces · titoli

Digitale e sport

Un serif con carattere, per i titoli e per l'accento corsivo della home.

Inter · testo

parlano la stessa lingua

Un sans-serif pulito, leggibile a lungo, per corpo del testo e interfaccia.

Dove di solito serve JavaScript, e come ho fatto senza

Il brief chiedeva HTML e CSS, con EmailJS come unica eccezione facoltativa. Ogni volta che sarebbe stato più comodo uno script, ho cercato prima una soluzione solo CSS.

1 Menu mobile

Una checkbox nascosta e un selettore `:checked`, senza una riga di script. Il dettaglio è nella prossima slide.

3 Anno nel footer

Scritto a mano nell'HTML e aggiornato una volta l'anno, invece di calcolarlo con uno script per un singolo numero.

2 Filtro del blog

Niente filtro dinamico via JS: categorie chiare nel testo e link diretti agli articoli, che restano indicizzabili anche senza JavaScript attivo.

4 Hover e transizioni

Tutto risolto con `transition` e `transform` in CSS: niente librerie di animazione da caricare.

IL MENU MOBILE

Un menu che si apre senza una riga di JavaScript

Il trucco si chiama "checkbox hack": una casella nascosta tiene lo stato, un selettore CSS fa il resto.

```
<!-- HTML -->
<input type="checkbox" id="nav-toggle"
      class="nav-toggle-checkbox">
<label class="nav-toggle" for="nav-toggle">
  <span></span><span></span><span></span>
</label>
<ul class="nav-menu"> ... </ul>

/* CSS */
.nav-toggle-checkbox:checked ~ .nav-menu {
  display: flex;
}
```

- Una checkbox nascosta tiene lo stato aperto/chiuso, al posto di una variabile JavaScript
- Il selettore `:checked ~` mostra il menu quando la checkbox è selezionata
- Funziona anche se il JavaScript del browser è disattivato
- La `label` è collegata alla checkbox con `for/id`, quindi resta utilizzabile anche da tastiera

SASS

Un CSS organizzato, non solo scritto

Con quattro pagine e componenti ripetuti, il CSS puro sarebbe diventato presto da riscrivere. Sass mi dà variabili e file separati, ma il risultato resta CSS normale.

_variables.scss

Colori, font, spaziature, breakpoint

_base.scss

Reset e stili di base, tipografia globale

_components.scss

Navbar, bottoni, card, hero, form, footer

style.scss

Punto di ingresso, importa tutti gli altri file

Il file compilato, `assets/css/style.css`, resta versionato nel repository:
GitHub Pages serve file statici, quindi il sito funziona anche senza un passaggio di build in produzione. Se tocco un file `.scss`, ricompilo e committo entrambi.

```
sass assets/scss/style.scss assets/css/style.css --style=expanded
```

BOOTSTRAP

Bootstrap, ma solo per la parte noiosa

Caricato da CDN e usato per il minimo indispensabile, così il sito non sembra un template qualunque.

Cosa uso di Bootstrap

- La griglia responsive, per allineare colonne e contenuti
- Poche utility di spaziatura e allineamento
- Nessun componente visivo preconfezionato

Cosa è mio

- Palette colori e tipografia, Fraunces più Inter
- Bottoni, card progetto, navbar, hero, form, footer
- Tutto il resto: è qui che il sito prende personalità

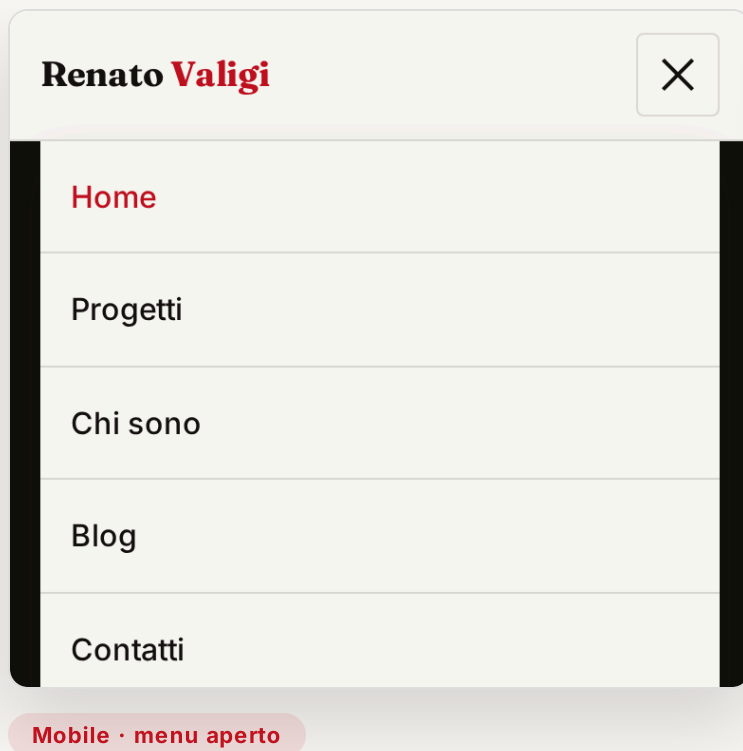
RESPONSIVE

Tre breakpoint, testati su dispositivi veri

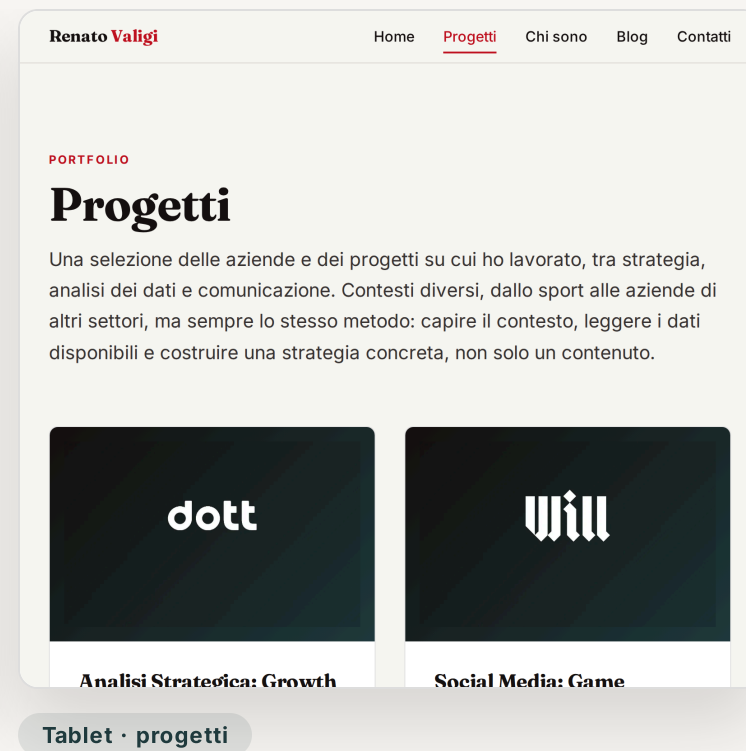
576, 768 e 992 pixel, definiti come variabili Sass e verificati non solo in DevTools ma su telefono e tablet reali.



Hero leggibile anche sotto i 400px, ritratto sotto al testo invece che a fianco.



Con la checkbox il menu si apre in pagina, voci una sotto l'altra, senza overlay.



A 768px la griglia passa da una a due colonne, senza forzare il layout desktop.

Piccoli dettagli che fanno la differenza

Nessuno di questi punti salta all'occhio, ma insieme cambiano com'è usare e trovare il sito.

Accessibilità

- Skip link per saltare al contenuto principale
- Stato : `focus-visible` ben visibile su link e bottoni
- Testo alternativo su tutte le immagini
- Struttura semantica: header, nav, main, footer
- Contrasto colori verificato sui toni scuri e chiari

SEO e condivisione

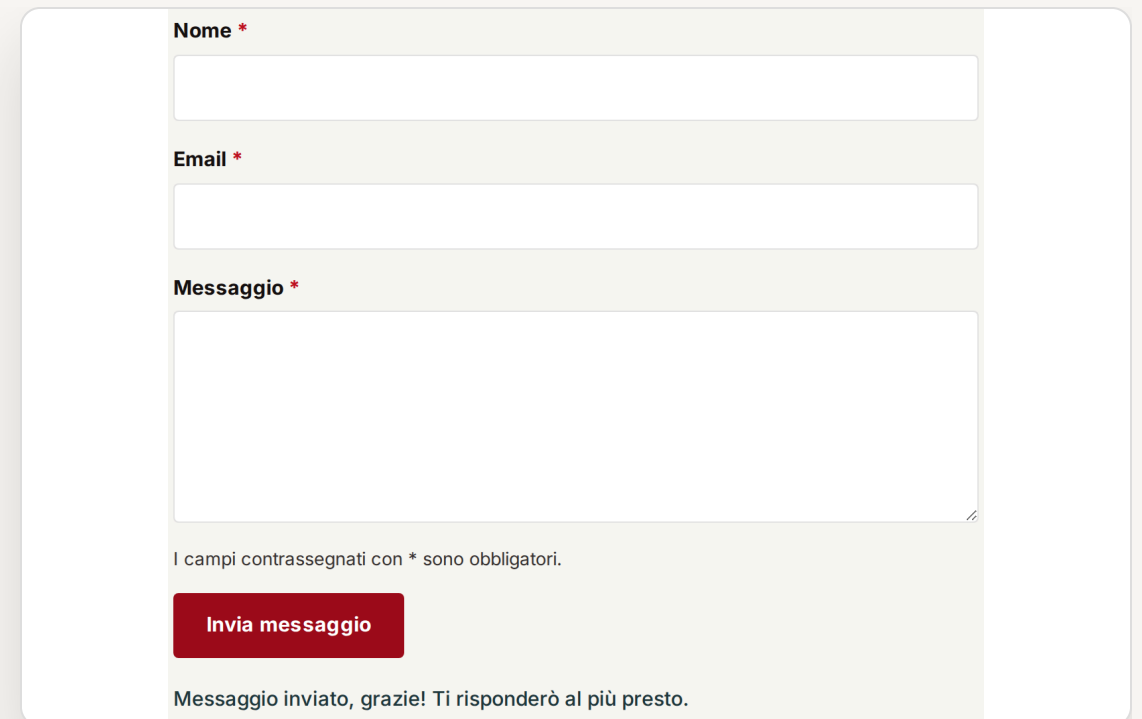
- Meta description diversa per ogni pagina
- Tag Open Graph e Twitter Card per la condivisione social
- Favicon e icona per la home su iOS
- Nessun indirizzo email o numero scritto in chiaro, per non farli raccogliere dai bot
- URL puliti, pronti per GitHub Pages

IL FORM CONTATTI

EmailJS, l'ultimo tassello

Nominato esplicitamente nel brief come opzione facoltativa. L'ho tenuto disattivato finché il resto del sito non era pronto, poi l'ho collegato davvero.

- Servizio Gmail collegato come mittente
- Template con gli stessi nomi dei campi del form, nome, email, messaggio, senza dover rinominare nulla
- La risposta (reply-to) va automaticamente a chi ha scritto
- Se EmailJS non fosse attivo, il form validerebbe comunque i campi e mostrerebbe un avviso, mai un errore silenzioso



The screenshot shows a contact form with three input fields and a submit button. The first field is labeled 'Nome *', the second 'Email *', and the third 'Messaggio *'. Below the fields, a note states 'I campi contrassegnati con * sono obbligatori.' (Fields marked with * are mandatory). The submit button is red and labeled 'Invia messaggio'. Below the button, a message reads 'Messaggio inviato, grazie! Ti risponderò al più presto.'

Test di invio reale, dal form pubblicato online

PUBBLICAZIONE

Dalla repository al sito online

Nessun passaggio di build in produzione: quello che è nel repository è quello che si vede online.

1 Repository pubblica

Codice versionato su GitHub, storico completo di ogni modifica.

2 Push su main

Ogni aggiornamento parte da un commit con un messaggio chiaro.

3 GitHub Pages

Attivato sulla root del branch main, senza build da configurare.

4 .nojekyll

Dice a Pages di non processare il sito con Jekyll, cartelle comprese.

RIEPILOGO

Dal brief al sito online

1

Analisi del brief

Definizione della struttura: cinque pagine, contenuti reali, vincolo HTML e CSS.

2

Architettura e identità

Struttura Sass, palette e tipografia, componenti riutilizzabili.

3

Sviluppo e responsive

Pagine, menu mobile solo CSS, verifica su tre breakpoint e dispositivi reali.

4

Rifinitura e pubblicazione

Accessibilità, SEO, attivazione di EmailJS e pubblicazione su GitHub Pages.

Stato attuale

Il sito è online, il form contatti invia davvero i messaggi via EmailJS e il repository è pubblico su GitHub.

IL PROGETTO

In due link

Il codice sorgente e la versione pubblicata: entrambi aggiornati a ogni modifica.

REPOSITORY

**[github.com/renatovaligi/
renato-valigi](https://github.com/renatovaligi/renato-valigi)**

Codice sorgente, cronologia dei commit e il README con tutte le scelte tecniche documentate.

→ Vedi su GitHub

SITO ONLINE

**[renatovaligi.github.io/
renato-valigi](https://renatovaligi.github.io/renato-valigi)**

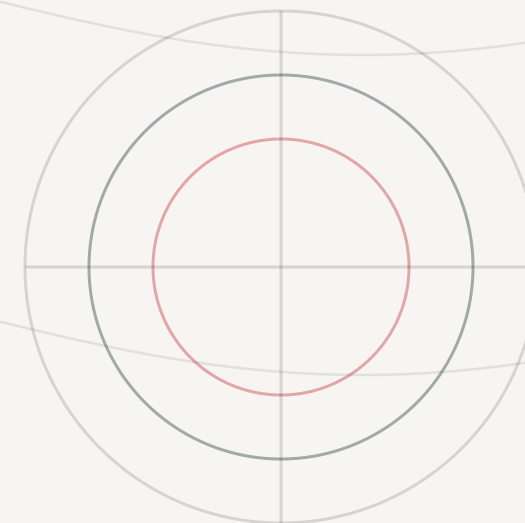
La versione pubblicata e navigabile, aggiornata automaticamente a ogni push su main.

→ Apri il sito

FINE PRESENTAZIONE

Grazie per l'attenzione

Per qualsiasi domanda sulle scelte del progetto, o per parlare di una collaborazione, sono raggiungibile ai contatti qui sotto.



RENATO VALIGI

Digital & Sports Communication Strategist

GitHub github.com/renatovaligi/renato-valigi

Sito renatovaligi.github.io/renato-valigi